



University of Asia Pacific

DEPARTMENT OF CSE

COURSE CODE : 402

COURSE TITLE : Operating System Lab

DATE OF SUBMISSION : 03/09/2026

SUBMITTED BY:

NAME : Fairoj Salim

STUDENT ID : 23101165

SECTION : C2

LEVEL : 4th Year 1st Semester

SUBMITTED TO:

Atia Rahman Orthi
LECTURER
Department of CSE, UAP

PROBLEM STATEMENT

Implement Priority Scheduling — both Non-Preemptive and Preemptive versions — for the following scenario where the number of processes = 5 and the process information is:

P_id	AT	BT	Priority
p1	0	3	3
p2	1	4	2
p3	2	6	4
p4	3	4	6
p5	5	2	10

A lower priority number means higher priority. Calculate CT, TAT, WT, Average TAT and Average WT for both Non-Preemptive and Preemptive Priority Scheduling. Also represent the process execution sequence for each, and compare the two approaches.

SOURCE CODE

```
p_id = ["p1", "p2", "p3", "p4", "p5"]
at = [0, 1, 2, 3, 5]
bt = [3, 4, 6, 4, 2]

priority = [3, 2, 4, 6, 10]

n = len(p_id)

np_ct = [0] * n
np_tat = [0] * n
np_wt = [0] * n
done = [0] * n

time = 0
completed = 0
np_exe_seq = []

while completed < n:
    highest = -1

    for i in range(n):
        if at[i] <= time and done[i] == 0:
            if highest == -1 or priority[i] < priority[highest]:
                highest = i

    if highest == -1:
        time = time + 1
```

```

else:

    np_exe_seq.append(p_id[highest])
    time = time + bt[highest]
    np_ct[highest] = time
    np_tat[highest] = np_ct[highest] - at[highest]
    np_wt[highest] = np_tat[highest] - bt[highest]
    done[highest] = 1
    completed = completed + 1

np_avg_tat = sum(np_tat) / n
np_avg_wt = sum(np_wt) / n

remaining = bt.copy()

p_ct = [0] * n
p_tat = [0] * n
p_wt = [0] * n

time = 0
completed = 0
p_exe_seq = []

while completed < n:
    highest = -1

    for i in range(n):
        if at[i] <= time and remaining[i] > 0:
            if highest == -1 or priority[i] < priority[highest]:
                highest = i

    if highest == -1:
        time = time + 1

    else:

        p_exe_seq.append(p_id[highest])
        remaining[highest] = remaining[highest] - 1
        time = time + 1

        if remaining[highest] == 0:
            p_ct[highest] = time
            completed = completed + 1

for i in range(n):
    p_tat[i] = p_ct[i] - at[i]
    p_wt[i] = p_tat[i] - bt[i]

p_avg_tat = sum(p_tat) / n
p_avg_wt = sum(p_wt) / n

print("\nNON-PREEMPTIVE PRIORITY:")
print("Execution Sequence:", end=" ")

for process in np_exe_seq:
    print(process, end=" ")

print("\n")
print(f'{{"PID":<8}}{{"AT":<6}}{{"BT":<6}}{{"Priority":<10}}{{"CT":<6}}{{"TAT":<6}}{{"WT":<6}}')

```

```

for i in range(n):

    print(
        f"{p_id[i]:<8}{at[i]:<6}{bt[i]:<6}"
        f"{priority[i]:<10}{np_ct[i]:<6}"
        f"{np_tat[i]:<6}{np_wt[i]:<6}"
    )

print("\nAverage TAT =", np_avg_tat)
print("Average WT =", np_avg_wt)

print("\nPREEMPTIVE PRIORITY:")
print("Execution Sequence:", end=" ")

for process in p_exe_seq:
    print(process, end=" ")

print("\n")
print(f'{"PID":<8}{AT":<6}{BT":<6}{Priority':<10}{CT':<6}{TAT":<6}{WT':<6}')

for i in range(n):

    print(
        f"{p_id[i]:<8}{at[i]:<6}{bt[i]:<6}"
        f"{priority[i]:<10}{p_ct[i]:<6}"
        f"{p_tat[i]:<6}{p_wt[i]:<6}"
    )

print("\nAverage TAT =", p_avg_tat)
print("Average WT =", p_avg_wt)

print("\nComparison:")
print(f'{"Algorithm":<25}{Avrg TAT":<15}{Avrg WT":<15}')
print(f'{"NON-PREEMPTIVE PRIORITY.':<25}{np_avg_tat:<15.2f}{np_avg_wt:<15.2f}")
print(f'{"PREEMPTIVE PRIORITY.':<25}{p_avg_tat:<15.2f}{p_avg_wt:<15.2f}")

```

OUTPUT

```
Priority Scheduling non preemptive & preemptive Imple... x
:
C:\Users\lab\PycharmProjects\PythonProject\.venv\Scripts\python.exe "C:\Users\lab\Pychar

NON-PREEMPTIVE PRIORITY:
Execution Sequence: p1 p2 p3 p4 p5

PID    AT    BT    Priority  CT    TAT  WT
p1      0     3     3         3     3    0
p2      1     4     2         7     6    2
p3      2     6     4        13    11    5
p4      3     4     6        17    14   10
p5      5     2    10        19    14   12

Average TAT = 9.6
Average WT  = 5.8

PREEMPTIVE PRIORITY:
Execution Sequence: p1 p2 p2 p2 p2 p1 p1 p3 p3 p3 p3 p3 p3 p4 p4 p4 p4 p5 p5

PID    AT    BT    Priority  CT    TAT  WT
p1      0     3     3         7     7    4
p2      1     4     2         5     4    0
p3      2     6     4        13    11    5
```

```
Run Priority Scheduling non preemptive & preemptive Imple... x
:
Average WT = 5.8

PREEMPTIVE PRIORITY:
Execution Sequence: p1 p2 p2 p2 p2 p1 p1 p3 p3 p3 p3 p3 p3 p4 p4 p4 p4 p5 p5

PID    AT    BT    Priority  CT    TAT  WT
p1      0     3     3         7     7    4
p2      1     4     2         5     4    0
p3      2     6     4        13    11    5
p4      3     4     6        17    14   10
p5      5     2    10        19    14   12

Average TAT = 10.0
Average WT  = 6.2

Comparison:
Algorithm      Avrg TAT      Avrg WT
NON-PREEMPTIVE PRIORITY: 9.60      5.80
PREEMPTIVE PRIORITY:      10.00      6.20

Process finished with exit code 0
```

DISCUSSION

In this experiment, Priority Scheduling was implemented in both its Non-Preemptive and Preemptive forms in Python, and the two were compared using Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT). In both versions, a lower priority number represents a higher-priority process, and at every scheduling decision the process with the lowest priority number among the available processes is chosen.

In Non-Preemptive Priority Scheduling, once a process starts running, it continues until it finishes, even if a higher-priority process arrives in the meantime. In Preemptive Priority Scheduling, the CPU is re-checked at every time unit, and if a newly arrived process has a higher priority (lower priority number) than the one currently running, the running process is immediately interrupted and pushed back into the ready queue.

From the output, Non-Preemptive Priority produced the execution sequence $p1 \rightarrow p2 \rightarrow p3 \rightarrow p4 \rightarrow p5$, with an average Turnaround Time of 9.6 and an average Waiting Time of 5.8. Preemptive Priority produced a longer sequence (p1, p2, p2, p2, p2, p1, p1, p3, p3, p3, p3, p3, p3, p4, p4, p4, p4, p5, p5), where p1 was interrupted after 1 time unit so that p2 (priority 2) could run first, and resumed only after p2 finished. This resulted in a higher average Turnaround Time of 10.0 and average Waiting Time of 6.2.

Non-Preemptive Priority vs Preemptive Priority: What the Results Show

- **Result here:** Non-Preemptive Priority performed better for this process set — lower average TAT (9.6 vs 10.0) and lower average WT (5.8 vs 6.2).
- **Why preemption hurt here:** p1 (priority 3) had already started and only had a slightly lower priority than p2 (priority 2). Interrupting p1 after just 1 unit delayed its completion by 4 units (WT 0 $\rightarrow$ 4), while p2 only gained 2 units of improved waiting (WT 2 $\rightarrow$ 0) — a net loss.
- **General rule:** Preemptive Priority is normally better when a much higher-priority process arrives while a low-priority process is running, since it avoids the low-priority process blocking an urgent one.
- **When Non-Preemptive wins:** When priority differences between processes are small, or a process has already run for a while before a slightly higher-priority process arrives, preemption adds context-switching overhead without a real benefit.
- **Common weakness:** Both versions can suffer from starvation — a low-priority process may wait indefinitely if higher-priority processes keep arriving; Preemptive Priority makes this worse since it can also interrupt a process that is close to finishing.

CONCLUSION

This experiment provided practical experience in implementing both Non-Preemptive and Preemptive Priority Scheduling using Python. Implementing the preemptive version required checking the ready queue at every single time unit instead of only when the CPU became free, which made it more complex than the non-preemptive version but allowed it to react instantly to higher-priority arrivals.

The results show that for this process set, Non-Preemptive Priority produced an average TAT of 9.6 and an average WT of 5.8, while Preemptive Priority produced a higher average TAT of 10.0 and average WT of 6.2, mainly because p1 was interrupted early by p2 despite only a small priority difference between them.

Non-Preemptive Priority vs Preemptive Priority: The Better Choice

- **For this process set:** Non-Preemptive Priority is the better choice, since it produced both a lower average TAT (9.6) and a lower average WT (5.8) than Preemptive Priority.
- **Why:** The priority difference between the interrupting and interrupted process (p2 vs p1) was too small to justify the cost of preemption here.
- **In general:** Preemptive Priority is better when high-priority, time-critical processes can arrive at any moment and must not be delayed by lower-priority processes already running — for example, in real-time or interactive systems.
- **Overall takeaway:** The better algorithm depends on the situation — Preemptive Priority favors responsiveness to urgent tasks, while Non-Preemptive Priority favors simplicity and lower overhead when priority gaps are small, as shown by this experiment.